

MindBridge 开发文档

项目概述

MindBridge 是一个基于 FastAPI 微服务架构的计算机视觉推理平台，整合了 YOLO 实例分割、LAST-ViT 特征匹配和 RealSense 深度估计三个核心能力。每个功能模块作为独立的微服务运行，通过 HTTP API 对外提供推理接口，并由统一的控制中心 (`main.py`) 编排流水线。

```
https://github.com/Xldln/multiservice-guidelines.git
```

以上代码库提供了一个非常不错的服务解耦skill，这将大大提升你的工作效率，同时也提供了case以供参考。

一、Shell 脚本说明

1. `build.sh` — Docker 镜像构建与 CLI 安装

用途： 从零构建开发/运行环境，包括 Docker 镜像和 `mindbridge` CLI 命令。

执行流程： 1. 基于 `Dockerfile` 构建名为 `mindbridge` 的 Docker 镜像 (CUDA 12.8 + Miniconda + 系统依赖) 2. 在 `~/local/bin/` 下创建 `mindbridge` 可执行脚本 3. 检查并将 `~/local/bin` 加入 `PATH`

用法：

```
# 首次构建环境
./build.sh

# 之后每次进入容器开发
mindbridge
```

`mindbridge` 命令行为： - 自动检查是否已有名为 `mindbridge` 的容器在运行 - 若不存在：以特权模式、GPU 全部挂载、host 网络模式启动容器，映射 X11 和 `/dev` 设备 - 进入容器的 bash shell，工作目录为宿主机当前目录

2. scripts/build_env.sh — Conda 环境构建

用途：在 Docker 容器内为每个微服务创建独立的 Conda 虚拟环境，安装各自的依赖。

架构设计：每个模型服务使用独立的 Conda 环境，原因是不同模型对 PyTorch/CUDA 版本、额外库（如 `pyrealsense2`）的要求可能冲突，独立环境可以做到完全隔离。

用法：

```
bash scripts/build_env.sh yolo          # 仅构建 YOLO 环境
bash scripts/build_env.sh realsense     # 仅构建 RealSense 环境
bash scripts/build_env.sh lastvit       # 仅构建 LAST-ViT 环境
bash scripts/build_env.sh all           # 构建全部三个环境（默认）
```

各环境依赖：

环境名	Python	核心依赖	用途
yolo	3.12	torch, ultralytics, opencv-python, fastapi, uvicorn, pydantic, lap	YOLO 实例分割推理
realsense	3.12	torch, pyrealsense2, omegaconf, opencv-python, fastapi, uvicorn, pydantic	RealSense 深度相机采集
lastvit	3.12	torch, torchvision, opencv-python, fastapi, uvicorn, pydantic, pillow	LAST-ViT 特征匹配推理

公共初始化 (`set_base`)： - 接受 Anaconda 服务条款 - 安装 DejaVu 字体（解决 OpenCV 中文渲染问题） - 安装 `bin/mind` 命令到系统 PATH（如果存在） - 设置 pip 镜像源为清华源

3. scripts/start_service.sh — 服务管理器

用途：统一管理所有微服务的启动、停止、重启和状态查看。

用法：

```
bash scripts/start_service.sh yolo      # 启动 YOLO 服务 → :8001
bash scripts/start_service.sh realsense # 启动 RealSense 服务 → :8000
bash scripts/start_service.sh all        # 同时启动两个服务
bash scripts/start_service.sh stop       # 停止所有服务
bash scripts/start_service.sh status     # 查看运行状态
bash scripts/start_service.sh restart    # 重启所有服务
```

架构细节： - 每个服务以 `nohup` 后台启动，日志写入 `logs/<name>.log` - PID 文件写入 `/tmp/mindbridge/<name>.pid`，用于重复启动检测和状态查询 - 启动前自动激活对应的 Conda 环境 - `stop` 命令会终止所有已记录 PID 的进程并清理 PID 文件 - 注册了 `SIGINT` / `SIGTERM` 信号处理，`Ctrl+C` 可优雅停止所有服务

当前支持的服务映射：

命令参数	Conda 环境	启动脚本
yolo	yolo	mindbridge/src/core/launch/service_InsenceSeg.py
realsense	realsense	mindbridge/src/core/launch/service_RealSense.py

注意： LAST-ViT 服务 (`service_Lastvit.py`) 在 `start_service.sh` 中未注册快捷命令，需要手动启动：`bash conda activate lastvit && python mindbridge/src/core/launch/service_Lastvit.py`

二、mindbridge/ 目录结构详解

整体架构图

```
mindbridge/
├── __init__.py                # 空文件，避免跨域依赖
├── src/
│   ├── main.py                # 【控制中心】编排 RealSense → YOLO 流水线
│   ├── MindBridgeClient.py    # 【HTTP 客户端】封装对微服务的请求
│   └── core/
│       ├── launch/            # —— 服务入口层 ——
│       │   ├── service_InsenceSeg.py    # YOLO 推理服务 (端口 8001)
│       │   ├── service_RealSense.py    # RealSense 深度服务 (端口 8000)
│       │   └── service_Lastvit.py      # LAST-ViT 特征匹配服务 (端口 8002)
│       ├── controller/        # —— 路由控制层 ——
│       │   ├── InstanceSegController.py # YOLO /infer/predict, /infer/predict/file
│       │   ├── RealSenseController.py   # /realsense/capture, /info, /shutdown
│       │   └── LastvitController.py      # LAST-ViT /infer/predict, /infer/predict/file
│       └── service/            # —— 业务逻辑层 ——
```

```

|   |   | InstanceSegmentInfer.py    # YOLO 模型加载、推理、结果解析
|   |   | RealsenseService.py       # RealSense 相机初始化、深度采集
|   |   | └ LastvitInfer.py          # LAST-ViT 模型定义、特征编码、相似度匹配
|   |
|   |   | schemas/                  # —— 数据模型层 ——
|   |   |   | YoloEntity.py          # Detection, PredictRequest/Response
|   |   |   | RealsenseEntity.py     # CaptureData, CaptureRequest/Response
|   |   |   | └ LastvitEntity.py     # TopKItem, StateItem, PredictRequest/Response
|   |
|   |   | tool/                     # —— 工具函数层 ——
|   |   |   | image.py               # base64 ↔ numpy 编解码、检测框绘制
|   |   |   | └ LastvitTools.py      # 图像预处理、相似度计算、可视化、Dashboard 上传
|   |
|   |   | config/                   # —— 配置文件 ——
|   |   |   | yolo-config.yaml       # YOLO 模型路径、置信度阈值
|   |   |   | realsense-config.yaml  # 相机分辨率、帧率、红外发射器
|   |   |   | └ last-vit-config.yaml # 模型权重路径、图信息文件、TopK
|
| └ models/
|   | config/
|   |   | └ yolo-config.yaml         # 备用 YOLO 配置
|   | weights/                       # 模型权重文件 (gitignore)
|   |   | yolo/
|   |   | └ lastvit/
|   | pkg/
|   |   | └ yolo/                   # Ultralytics 源码 (可编辑安装)

```

2.1 src/ — 核心源码

src/main.py — 控制中心

职责：编排 RealSense 采集 → YOLO 推理的完整流水线。

核心流程： 1. 等待 RealSense (8000) 和 YOLO (8001) 两个服务就绪（健康检查，最多等待 60 秒） 2. 循环执行：从 RealSense 抓取一帧 → 发送到 YOLO 推理 → 显示/保存结果 3. 支持 OpenCV 窗口实时预览（按 ESC 退出） 4. 支持无头模式（`--no-show`）、限制帧数（`--max-frames`）、保存结果（`--out-dir`）

命令行参数：

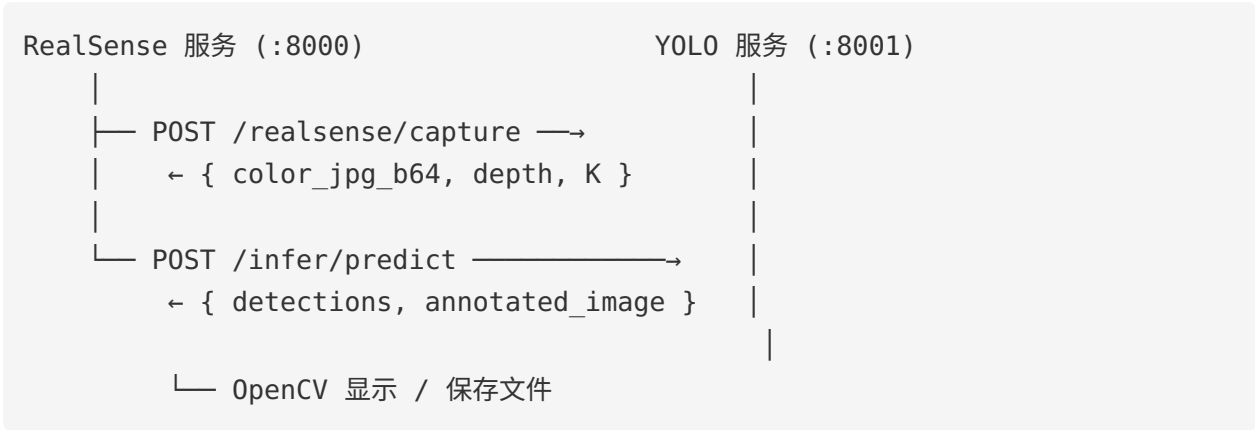
```

--realsense-url    RealSense 服务地址（默认 http://127.0.0.1:8000）
--yolo-url         YOLO 服务地址（默认 http://127.0.0.1:8001）

```

--no-show	无头模式，不显示 OpenCV 窗口
--max-frames N	最大处理帧数（默认无限）
--out-dir DIR	标注结果图保存目录
--log-interval N	日志输出间隔帧数（默认 30）

数据流：



src/MindBridgeClient.py — HTTP 客户端

职责：封装对微服务 API 的 HTTP 调用，供控制中心使用。

方法：

方法	HTTP	端点	说明
health()	GET	/health (两个服务)	检查 RealSense 和 YOLO 服务状态
capture()	POST	/realsense/capture	从 RealSense 抓取一帧
predict(image_b64, request_id)	POST	/infer/predict	发送图片到 YOLO 推理

2.2 core/launch/ — 服务入口层

每个文件是一个独立的 FastAPI 应用入口，负责：端口绑定、lifespan 生命周期管理、路由注册。

设计原则：launch/ 只做"启动"，不包含任何业务逻辑。

service_InstanceSeg.py — YOLO 推理服务

- 端口: 8001
- Conda 环境: `yolo`
- lifespan: 启动时加载 YOLO 模型, 关闭时释放资源
- 注册路由: `InstanceSegController.infer_router`
- 健康检查: `GET /health`

service_RealSense.py — RealSense 深度服务

- 端口: 8000
- Conda 环境: `realsense`
- lifespan: 启动时初始化相机、预热 30 帧
- 注册路由: `RealSenseController.realsense_router`
- 健康检查: `GET /health`

service_Lastvit.py — LAST-ViT 特征匹配服务

- 端口: 8002
- Conda 环境: `lastvit`
- lifespan: 启动时加载 LAST-ViT 模型和类别中心
- 注册路由: `LastvitController.infer_router`
- 健康检查: `GET /health`

2.3 core/controller/ — 路由控制层

每个 Controller 定义 FastAPI `APIRouter`, 负责: 1. HTTP 接口定义 (路径、方法、参数) 2. 请求 Schema 校验 3. 调用 Service 层执行业务逻辑 4. 响应序列化

设计原则: `controller/` 只做"路由和校验", 不包含模型加载、图像处理等业务逻辑。

InstanceSegController.py

端点	方法	说明
<code>/infer/predict</code>	POST	JSON body 传入 base64 图片, 返回检测结果
<code>/infer/predict/file</code>	POST	multipart/form-data 上传图片文件

请求参数 (`PredictRequest`): - `image_b64`: BGR 图像的 base64 编码 (必填) - `conf`: 置信度阈值 (可选, 覆盖服务端默认值) - `return_masks`: 是否返回分割掩码 -

`return_annotated_image`：是否返回标注后的图片 - `tracker`：跟踪器配置（默认 `bytetrack.yaml`）

响应（`PredictResponse`）：- `detections[]`：检测目标列表（id, label, score, bbox, mask_png_b64）- `annotated_image_b64`：标注后的 JPG base64 - `elapsed_sec`：推理耗时

`RealSenseController.py`

端点	方法	说明
<code>/realsense/capture</code>	POST	采集一帧（彩色 JPG + 深度 PNG base64）
<code>/realsense/info</code>	GET	返回相机参数（内参 K、基线 baseline、分辨率）
<code>/realsense/shutdown</code>	POST	关闭相机，释放资源

`LastvitController.py`

端点	方法	说明
<code>/infer/predict</code>	POST	JSON body 传入 base64 图片，返回特征匹配结果
<code>/infer/predict/file</code>	POST	multipart/form-data 上传图片文件

2.4 `core/service/` — 业务逻辑层

Service 层包含核心业务逻辑：模型加载、推理计算、相机控制等。

设计原则： `service/` 独立于 HTTP 协议，可被 Controller 调用，也可被其他模块直接使用。

`InstanceSegmentInfer.py` — YOLO 推理引擎

核心类： `YOLOInfer`

功能： - 从 YAML 配置文件读取模型路径和置信度阈值 - 加载 Ultralytics YOLO 模型（支持检测、分割、分类任务） - 支持目标跟踪（ByteTrack） - 解析推理结果：边界框、类别、置信度、分割掩码 - 生成标注后的可视化图像

推理流程： 1. base64 → BGR numpy 数组 2. 调用 `model.track()` 进行推理+跟踪 3. 解析 `results[0].boxes` 获取检测框 4. 解析 `results[0].masks` 获取分割掩码 5. 调用 `result[0].plot()` 生成标注图 6. 构造 `PredictResponse` 返回

RealsenseService.py — RealSense 相机服务

核心类： RealsenseService

功能： - 通过 pyrealsense2 初始化 RealSense 深度相机 - 配置彩色流 (BGR8) 和深度流 (Z16) - 自动对齐深度帧到彩色帧坐标系 - 预热 30 帧等待自动曝光稳定 - 获取相机内参矩阵 K 和立体基线

捕获数据 (CaptureData)： - color_bgr：彩色图 (H,W,3) uint8 - depth_m：深度图 (H,W) float32 (米) - depth_u16：深度图 (H,W) uint16 (毫米) - K：相机内参 (3,3) - baseline：立体基线 (米)

LastvitInfer.py — LAST-ViT 推理引擎

核心类： LastvitInfer

模型架构： - LASTViTVisionEncoder：基于 ViT-B/16 的视觉编码器 - 特有机制：频域 Token 选择 (Frequency-domain Token Selection)，通过对 Transformer 输出的 token 做 FFT → 高斯平滑 → 高频差异筛选，选择信息量最大的 token - 输出 512 维 L2 归一化特征向量

功能： - 加载 LAST-ViT checkpoint 权重 - 加载类别中心特征 (从 graph_info.json) - 图像预处理：CLIP 标准预处理 (resize + normalize) - 特征编码 + 余弦相似度匹配 - 返回 Top-K 匹配结果

2.5 core/schemas/ — 数据模型层

使用 Pydantic v2 定义所有 API 的请求/响应数据结构。

文件说明：

文件	主要模型	用途
YoloEntity.py	PredictRequest , PredictResponse , Detection	YOLO 推理请求/响应
RealsenseEntity.py	CaptureData , CaptureRequest , CaptureResponse , CameraInfoResponse	RealSense 采集请求/响应
LastvitEntity.py	PredictRequest , PredictResponse , TopKItem , StateItem	LAST-ViT 特征匹配请求/响应

加新 Schema 的规范： 1. 在 schemas/ 下新建 XxxEntity.py 2. 所有模型继承 pydantic.BaseModel 3. 使用 Field() 添加描述和校验约束 4. 在 __init__.py 中导出 (当前 schemas/__init__.py 已导出 YoloEntity 和 LastvitEntity)

2.6 core/tool/ — 工具函数层

存放无状态的纯工具函数，供多个模块复用。

image.py

函数	说明
<code>decode_bgr_from_base64(b64)</code>	base64 解码为 BGR numpy 数组
<code>encode_bgr_to_base64_jpg(img, quality)</code>	BGR numpy 编码为 base64 JPG
<code>encode_mask_to_base64_png(mask)</code>	uint8 mask 编码为 base64 PNG
<code>_draw_detections(bgr, detections)</code>	在图像上绘制检测框和标签

LastvitTools.py

函数	说明
<code>decode_image_b64_to_pil(b64)</code>	base64 → PIL RGB
<code>decode_image_b64_to_bgr(b64)</code>	base64 → OpenCV BGR
<code>apply_padding_head(image)</code>	顶部 25% 区域填充黑色（消除头部干扰）
<code>parse_center_feature(value)</code>	解析类别中心特征向量并 L2 归一化
<code>calculate_similarity(feats, centers, topk)</code>	余弦相似度计算，返回 best + topk
<code>create_state_diagram(states, current_id)</code>	生成状态节点环形图
<code>build_visualization_frame(img, result, states)</code>	拼接推理结果 + 状态图
<code>upload_visualization_frame(...)</code>	编码帧并上传到 Dashboard

2.7 core/config/ — 配置文件

所有配置使用 YAML 格式，支持通过环境变量覆盖路径：

```
export YOLO_CONFIG=/custom/path/yolo-config.yaml
export REALSENSE_CONFIG=/custom/path/realsense-config.yaml
export LASTVIT_CONFIG=/custom/path/last-vit-config.yaml
```

yolo-config.yaml

```
yolo:
  model_path: /workspace/mindbridge/models/weights/yolo/best_yolo11m_pencilbag.pt
  score_threshold: 0.4
```

realsense-config.yaml

```
camera:
  width: 640
  height: 480
  fps: 30
  disable_emitter: 0    # 0=关闭红外发射器, 1=开启
runtime:
  scale: 1.0
  z_far: 10.0
visualization:
  show: true
```

last-vit-config.yaml

```
model:
  checkpoint: /workspace/mindbridge/models/weights/lastvit/checkpoint.pth
  graph_info_file: /workspace/mindbridge/models/weights/lastvit/graph_info.json
  topk: 5
```

2.8 models/ — 模型资源

路径	用途
models/weights/yolo/	YOLO 训练好的 .pt 权重文件（gitignore，需自行放入）
models/weights/lastvit/	LAST-ViT checkpoint 和 graph_info.json
models/pkg/yolo/	Ultralytics 源码，通过 <code>pip install -e .</code> 可编辑安装
models/config/	备用 YOLO 配置文件

三、架构设计原则

三层微服务架构



依赖方向（严格遵守，不可反向）：

```
launch/ → controller/ → service/ & schemas/ & tool/
```

关键设计决策

- 1. **独立 Conda 环境**：每个微服务使用独立环境，避免依赖冲突。例如 `pyrealsense2` 只在 `realsense` 环境中安装。
- 2. **init.py 刻意留空**：`controller/`、`service/`、`config/` 的 `__init__.py` 均为空文件（或仅含注释），避免导入时拉入设备相关依赖。各服务入口直接从具体模块导入。
- 3. **配置驱动**：所有模型路径、阈值、相机参数通过 YAML 文件管理，支持环境变量覆盖。
- 4. **base64 编解码传输**：所有图像数据以 base64 编码在 HTTP body 中传输，兼容 JSON API。
- 5. **Service 层无状态**：Service 类在 lifespan 中初始化一次并缓存为全局变量，所有请求复用同一实例。

四、开发流程步骤

步骤 1：环境准备

```
# 1. 克隆项目
git clone <repo-url>
cd kernel-robot

# 2. 构建 Docker 镜像 + CLI
./build.sh

# 3. 进入容器
mindbridge

# 4. 构建 Conda 环境（在容器内执行）
bash scripts/build_env.sh all
```

步骤 2：放置模型权重

将训练好的模型文件放入对应目录：

```
mindbridge/models/weights/yolo/best.pt          # YOLO 权重
mindbridge/models/weights/lastvit/checkpoint.pth # LAST-ViT 权重
mindbridge/models/weights/lastvit/graph_info.json # LAST-ViT 类别中心
```

步骤 3：修改配置

编辑 `mindbridge/src/core/config/` 下的 YAML 文件，修改模型路径和参数：

```
vim mindbridge/src/core/config/yolo-config.yaml
vim mindbridge/src/core/config/realsense-config.yaml
vim mindbridge/src/core/config/last-vit-config.yaml
```

或通过环境变量覆盖：

```
export YOLO_CONFIG=/custom/config.yaml
```

步骤 4：启动服务

```
# 启动 RealSense 深度服务
bash scripts/start_service.sh realsense
```

```
# 启动 YOLO 推理服务
bash scripts/start_service.sh yolo

# 查看状态
bash scripts/start_service.sh status
```

步骤 5：运行控制中心

```
# 交互模式（显示窗口）
python mindbridge/src/main.py

# 无头模式，处理 100 帧并保存结果
python mindbridge/src/main.py --no-show --max-frames 100 --out-dir ./output
```

步骤 6：测试 API

```
# 健康检查
curl http://localhost:8000/health
curl http://localhost:8001/health

# RealSense 采集
curl -X POST http://localhost:8000/realsense/capture

# YOLO 推理
curl -X POST http://localhost:8001/infer/predict \
  -H "Content-Type: application/json" \
  -d '{"image_b64": "<base64_image_data>"}'
```

步骤 7：停止服务

```
bash scripts/start_service.sh stop
```

五、如何添加新服务

以添加一个"文本推理"服务为例：

1. 创建 Schema: `mindbridge/src/core/schemas/TextEntity.py`

```
from pydantic import BaseModel, Field

class TextRequest(BaseModel):
    text: str = Field(..., description="输入文本")

class TextResponse(BaseModel):
    status: str
    result: str
```

2. 创建 Service: `mindbridge/src/core/service/TextInfer.py`

```
class TextInfer:
    def __init__(self, config_path):
        # 加载模型
        pass

    def predict(self, req):
        # 推理逻辑
        pass
```

3. 创建 Controller: `mindbridge/src/core/controller/TextController.py`

```
from fastapi import APIRouter
from mindbridge.src.core.schemas.TextEntity import TextRequest, TextResponse
from mindbridge.src.core.service.TextInfer import TextInfer

text_router = APIRouter(prefix="/text", tags=["Text"])
engine = None

def init_engine(config_path):
    global engine
    engine = TextInfer(config_path)

@text_router.post("/predict", response_model=TextResponse)
def predict(body: TextRequest):
    return engine.predict(body)
```

4. 创建 Launch: `mindbridge/src/core/launch/service_Text.py`

```
import uvicorn
from fastapi import FastAPI
from mindbridge.src.core.controller.TextController import text_router, init_engine

app = FastAPI()
app.include_router(text_router)

@app.on_event("startup")
async def startup():
    init_engine("config.yaml")

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8003)
```

5. 注册到 `start_service.sh`

在 `scripts/start_service.sh` 的 case 分支中添加:

```
text)
    start_service "text" "text_env" "mindbridge/src/core/launch/service_Text.py"
;;
```

6. 添加到 `build_env.sh`

在 `scripts/build_env.sh` 中添加 `build_text()` 函数并在 case 分支注册。

六、调试与故障排查

查看服务日志

```
tail -f logs/yolo.log
tail -f logs/realsense.log
```

检查服务状态

```
bash scripts/start_service.sh status
```

手动启动（前台调试）

```
conda activate yolo
python mindbridge/src/core/launch/service_InsenceSeg.py
```

检查 GPU 可用性

```
nvidia-smi
python -c "import torch; print(torch.cuda.is_available())"
```

检查 RealSense 相机

```
conda activate realsense
python tests/realsense_s.py
```

七、项目文件清单

路径	类型	说明
build.sh	Shell	Docker 构建 + CLI 安装
scripts/build_env.sh	Shell	Conda 环境构建
scripts/start_service.sh	Shell	服务管理器
Dockerfile	Docker	CUDA 12.8 基础镜像
pyproject.toml	Python	项目元数据、依赖、工具配置
mindbridge/src/main.py	Python	控制中心
mindbridge/src/MindBridgeClient.py	Python	HTTP 客户端
mindbridge/src/core/launch/*.py	Python	服务入口（3 个）
mindbridge/src/core/controller/*.py	Python	路由控制器（3 个）
mindbridge/src/core/service/*.py	Python	业务逻辑（3 个）
mindbridge/src/core/schemas/*.py	Python	数据模型（3 个）
mindbridge/src/core/tool/*.py	Python	工具函数（2 个）
mindbridge/src/core/config/*.yaml	YAML	配置文件（3 个）

路径	类型	说明
tests/realSense_s.py	Python	RealSense 连通性测试
mindbridge/models/	目录	模型权重和源码包
README.md / README-zh.md	Markdown	项目说明